The Art Framer — Security Report (24 Oct 2026)

Oct 24, 2026 · @Theartframer

Verdict

The site is not hacked, carries no malware, and the hardening added earlier holds. The scan found one serious leftover: a script named setup-ssh-key.php sat in the website root since 23 May, open to anyone on the internet, and each visit rewrote the hosting account's SSH keys. There is no sign it was abused. The fix is prepared in pull request 429 and waits for the owner's merge.

Three read-only scripts ran on the live server on 24 Oct 2026 (tools/diag-security.php, -2.php, -3.php). Nothing was changed by the scan itself.

What was checked and passed

Every check below ran against the live server, not a copy.

Area	Check	Result
Malware	Theme, plugins, uploads and website root scanned for eval, base64 loaders, shells, hidden iframes, injected JavaScript	Clean. One deprecated function in the unused Transposh plugin, not malicious
Core files	WordPress 7.1.3 files compared with the official checksums	Intact. The one difference, wp-cron.php, is Hostinger's own patch
Database	Options table checked for injected scripts, unknown scheduled tasks, rogue redirects	Clean
Users	Administrator list and last logins reviewed	No unexpected accounts created. 5 admins, see decisions below
Login	Login throttling, rate limiting, Turnstile, masked error messages, application passwords admin-only	All on

Area	Check	Result
Attack surface	XML-RPC off, user enumeration blocked, REST user listing blocked, file editing disabled	All closed
Files	Uploads directory listing, debug.log readable from the web, wp-config.php exposure	All blocked
Headers	Content-Security-Policy (report-only on checkout), X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, HSTS	All present
TLS	Certificate chain and expiry	Valid until 17 Jan 2027
Embeds	Third-party iframes and scripts on public pages	Only Google Maps, GA4, Meta pixel, Instagram feed. All expected

Issues found

One high-severity leftover and a handful of hygiene items. None shows signs of having been exploited.

#	Severity	Issue	What it means in plain words
1	High	setup-ssh-key.php in the website root (23 May 2026), answered HTTP 200 to anyone	Each visit overwrote the hosting account's ~/.ssh/authorized_keys with the GitHub deploy key and printed the file. The key is fixed, so a stranger could not add their own key, but anyone could reset your SSH access and read the server home path and public key. Left over from the May deploy setup

#	Severity	Issue	What it means in plain words
2	Medium	create_autologin_<hash>.php in the website root (23 Jan 2026)	Hostinger's hPanel auto-login helper. It signs a browser in as a named account with no password. The account it names no longer exists and the file returns 404, but it should never have stayed nine months
3	Medium	Open registration on the WordPress login page (users_can_register = 1)	Anyone could create a subscriber account through wp-login.php. The form is rate-limited, so the risk is spam accounts, not takeover. WooCommerce My Account registration is a separate setting and stays on
4	Low	wp-content/debug.log at 78 MB	Hostinger forces debug mode on even though wp-config.php says off. The file is not readable from the web. It wastes disk and I/O only
5	Low	5 administrator accounts, 2 never used from a browser	Ayushi has never logged in. Dinesh has never logged in from a browser but holds one application password. Every unused admin is a password that can leak
6	Low	41 inactive plugins on disk	Inactive plugin code still sits on the server and some of it can be reached directly. Each one is attack surface for nothing in return
7	Info	.htaccess.bk in the website root	A stale LiteSpeed backup block. Harmless clutter

#	Severity	Issue	What it means in plain words
8	Info	No Redis or Memcached behind the object cache	Not a security issue. Noted because it is the reason uncached pages still take about 1.3 s

The fix prepared (pull request 429)

Issues 1, 2, 3, 4 and 7 are fixed by one new tool, tools/security-cleanup.php, which runs on every deploy and does nothing when there is nothing left to do. Nothing is applied until the owner merges pull request 429.

Step	What it does	Safety net
Quarantine the three files	Moves setup-ssh-key.php, create_autologin_<hash>.php and .htaccess.bk out of the website root into wp-content/af-quarantine/, renamed to .txt so they can never run again	Copies are kept. The folder has a deny-all .htaccess. The deploy key itself stays installed, so GitHub deploys keep working
Close open registration	Sets users_can_register to 0	WooCommerce My Account registration is untouched
Trim debug.log	Empties the file once it passes 50 MB	The file is write-only hygiene; nothing depends on its history
Purge cache	Clears LiteSpeed if any file moved	Pages re-cache on the next visit

Tested against a fixture holding the three files and a 57 MB log: the first run moves and fixes everything, the second run reports nothing to do.

After the merge, one manual run of the Deploy workflow is needed, because the automatic deploy ignores the tools and workflow folders. I will trigger it and then re-run the security scan.

Decisions needed from the owner

Issues 5 and 6 and the debug-mode setting live in dashboards, not in code, so they need the owner's hand.

- [ ] **Administrators.** In WordPress → Users, demote or remove Ayushi (never logged in). Review Dinesh: never logged in from a browser, but holds one application password; revoke it if nobody uses it. Keep only the people who need full control.
- [ ] **Inactive plugins.** In WordPress → Plugins → Inactive, delete the plugins that are not coming back. 41 are on disk; the four active ones (Elementor, LiteSpeed Cache, WooCommerce, WooCommerce Square) stay.
- [ ] **Debug mode.** In Hostinger hPanel, turn off WordPress debug / display errors for this site so debug.log stops growing and PHP notices are not generated on every page.
- [ ] **Optional, speed not security.** Enable Redis object cache in hPanel. The site already has object caching switched on with no backend behind it.

If you want, say which of these you approve and I will prepare the exact steps or, where a tool can do it safely, a script shown to you first.

Verification after deploy

Once pull request 429 is merged and deployed, the same read-only scan runs again and the report is updated with the result.

Check	Expected after deploy
setup-ssh-key.php, create_autologin_<hash>.php, .htaccess.bk in website root	Gone (HTTP 404), copies present in wp-content/af-quarantine/ as .txt
users_can_register	0
wp-content/debug.log	Under 1 MB
GitHub Actions deploy	Still succeeds (deploy key unchanged)
Full diag-security.php run	Same pass list as above, zero docroot leftovers

MCP and API connections

No key, token or password has leaked into the repository, its history, or the public pages of the site. The real exposure on this side is that the GitHub repository is public, so every workflow log is readable by anyone, and the security scans wrote admin usernames, login IP addresses and server paths into those logs. The three scan logs from today were deleted at 07:15 UTC; making the repository private closes the rest.

Connection	Credential and where it lives	What it can do	Finding
GitHub (connector)	Owner's account, scoped to this one repository	Read and write branches, open pull requests, run workflows. Direct pushes to main are blocked; changes go through reviewed pull requests	Repository is public (see below). 3 collaborators: owner (admin), Yash-devl (write), yashwinquest-cell (write)

Connection	Credential and where it lives	What it can do	Finding
Hostinger SSH deploy	Private key, host, port and user as GitHub Actions secrets	Copy files and run WP-CLI on the live server. 60 workflows use it; all are manual except the deploy on a push to main and 4 self-updating ones on claude/ branches. Only collaborators with write access can start them; no workflow runs on pull requests from strangers; secrets are masked in logs	OK

Connection	Credential and where it lives	What it can do	Finding
Google: YouTube Data, YouTube Analytics, Search Console, GA4	OAuth client, refresh token and API key held only as environment secrets in the separate "SB - Art Framer" session	Read channel and traffic data; write access to YouTube only when the owner approves an edit	Confirmed 24 Oct: values exist only as environment variables, 0 copies in any file, backup or git history; no output ever printed a value. The refresh token carries three write scopes (youtube, webmasters, business.manage) beside two read-only ones. The API key is not referrer-restricted. Both are owner decisions below
Meta Ads (connector)	OAuth through claude.ai	Read and create campaigns on 2 active ad accounts (USD and INR). Everything is created paused by rule	OK. No pixel dataset exists yet
Cloudflare Developer Platform (connector) and CLOUDFLARE_API_TOKEN, R2 secrets in Actions	Connector OAuth; token and R2 keys as Actions secrets	Workers, KV, R2 on the Cloudflare account	The site is served by Hostinger, not Cloudflare. Connector is deprecated and unused. Remove the token and R2 secrets unless R2 is still planned

Connection	Credential and where it lives	What it can do	Finding
Turnstile keys	Actions secrets	Bot check on forms	OK
YT_COOKIES	Actions secret used by the scheduled fetch-reels workflow	A cookie export of a signed-in Google account, used to download reels	A cookie export is a full session credential for that Google account. Confirm which account it belongs to; if reel fetching is no longer needed, delete the secret and the schedule
Canva, Gmail, Lovable	Connector OAuth	Not usable: Canva needs reconnect, Gmail and Lovable never finished connecting	Not a security issue. Remove the ones not in use

The public repository. Anyone can read the theme code, the workflows, the server path inside the Hostinger account, the committed reports, and every workflow run log, including scan output that lists administrator usernames and their login IP addresses. That is enough for a targeted login attack and it named the exploitable file before it was removed. One setting fixes all of it: GitHub → Settings → General → Danger Zone → Change visibility → Private. Deploys keep working; the workflows run in well under the free plan's private-repository minutes.

Decisions needed from the owner, API side

- [ ] Make the repository private (the one that matters most).
- [ ] Confirm the third collaborator, yashwinquest-cell, is Yash's own second account; otherwise remove it.
- [ ] Delete the Cloudflare token and R2 secrets from GitHub Actions if Cloudflare is not coming back; disconnect the deprecated Cloudflare connector.

- [ ] Decide whether the reels fetch is still needed; if not, delete the YT_COOKIES secret and the scheduled workflow.
- [ ] Google least privilege, optional: the current refresh token can edit YouTube videos, resubmit sitemaps and manage the Business Profile. Reporting needs none of that. If wanted, a second read-only token (youtube.readonly, webmasters.readonly, yt-analytics.readonly, analytics.readonly) can serve the reports and the write token stays for approved edits only.
- [ ] In Google Cloud Console (project 771757105416) → APIs & Services → Credentials, restrict the YouTube API key to the YouTube Data API v3 only. It is not referrer-restricted today, which is expected for server-side use, but it should not be able to call other Google APIs.